

Tomada de Decisão com Condicionais (if/else e Ternário)

1. Estrutura básica if

JavaScript

```
let idade = 20;

if (idade >= 18) {
  console.log("Maior de idade");
}
```

2. Adicionando o else

JavaScript

```
let idade = 15;

if (idade >= 18) {
  console.log("Maior de idade");
} else {
  console.log("Menor de idade");
}
```

3. Usando else if para múltiplas condições

Note que a ordem importa: verificamos primeiro a idade mais avançada ou ajustamos as faixas para que a lógica faça sentido.

JavaScript

```
let idade = 65;

if (idade >= 60) {
  console.log("Idoso");
} else if (idade >= 18) {
  console.log("Maior de idade");
} else {
  console.log("Menor de idade");
}
```

4. Teste de validação Truthy (Número 0)

Quando você passa o número 0 em um if, o bloco **não é executado**.

- **O que acontece:** O JavaScript (e muitas outras linguagens) interpreta o número 0 como false.
- **Por que?** No conceito de *Truthy* e *Falsy*, o 0, null, undefined, "" (string vazia) e NaN são sempre convertidos para **false**.

JavaScript

```
if (0) {
  console.log("Isso não vai aparecer");
} else {
  console.log("O número 0 é avaliado como falso (falsy)");
}
```

5. Operador Ternário

O operador ternário é uma forma compacta de escrever o `if/else`, seguindo a estrutura: `condição ? expr1 : expr2`.

JavaScript

```
let idade = 20;  
console.log(idade >= 18 ? "Maior de idade" : "Menor de idade");
```

Roteamento de Múltiplos Caminhos (switch)

1. Switch para validar `diaDaSemana`

JavaScript

```
let diaDaSemana = 3;  
  
switch (diaDaSemana) {  
  case 1:  
    console.log("Domingo");  
    break;  
  case 2:  
    console.log("Segunda-feira");  
    break;  
  case 3:  
    console.log("Terça-feira");  
    break;  
  case 4:  
    console.log("Quarta-feira");  
    break;  
  case 5:  
    console.log("Quinta-feira");  
    break;  
  case 6:  
    console.log("Sexta-feira");  
    break;  
  case 7:  
    console.log("Sábado");  
    break;  
  default:  
    console.log("Dia inválido");  
}
```

2. Removendo o `break` (efeito *fall-through*)

Exemplo (removendo o `break` do case 3):

JavaScript

```
let diaDaSemana = 3;
```

```
switch (diaDaSemana) {  
  case 1:  
    console.log("Domingo");  
    break;  
  case 2:  
    console.log("Segunda-feira");  
    break;  
  case 3:  
    console.log("Terça-feira");  
  case 4:  
    console.log("Quarta-feira");  
    break;  
  default:  
    console.log("Dia inválido");  
}
```

O que aconteceu?

Quando você remove o **break**, o JavaScript **continua executando os próximos cases**, mesmo que não correspondam.

Resultado no console:

```
JavaScript  
Terça-feira  
Quarta-feira
```

Isso é chamado de **fall-through**.

3. E se **diaDaSemana = "1"** (string)?

```
JavaScript
```

```
let diaDaSemana = "1";
```

O que acontece?

O **switch** usa **comparação estrita (===)**, então:

- **"1"** (string) **≠ 1** (number)

```
JavaScript
```

```
Resultado: Dia inválido
```

Como corrigir isso?

Você pode converter o valor antes:

```
JavaScript
```

```
let diaDaSemana = Number("1");
```

ou:

```
JavaScript
```

```
switch (Number(diaDaSemana)) {
```

4. Teste com valor inválido (ex: 8)

JavaScript

```
let diaDaSemana = 8;
```

Resultado:

JavaScript

Dia inválido

Automatização com Laços de Repetição (for e while)

1. Crie um for que conte de 1 a 10 imprimindo os números.

Neste laço, começamos com `i` valendo 1, continuamos enquanto `i` for menor ou igual a 10, e somamos 1 (`i++`) a cada repetição.

JavaScript

```
for (let i = 1; i <= 10; i++) {  
  console.log(i);  
}
```

2. Modifique o exercício 1 para contar regressivamente de 10 até 1.

Aqui invertemos a lógica: começamos no 10, o laço continua enquanto `i` for maior ou igual a 1, e subtraímos 1 (`i--`) a cada repetição.

JavaScript

```
for (let i = 10; i >= 1; i--) {  
  console.log(i);  
}
```

3. Crie um while que duplique o valor de uma variável saldo = 10 até que passe de 100.

O `while` é ideal quando não sabemos exatamente quantas vezes o laço vai rodar, apenas a condição de parada. Ele vai continuar rodando *enquanto* o saldo for menor ou igual a 100.

JavaScript

```
let saldo = 10;  
  
while (saldo <= 100) {  
  saldo = saldo * 2; // Você também pode escrever: saldo *= 2;  
  console.log("Saldo atual:", saldo);  
}  
  
console.log("Laço encerrado. Saldo final passou de 100:", saldo);
```

4. Utilize break para parar um loop for de 1 a 20 assim que ele bater no número 13.

O `break` funciona como um "botão de emergência" que encerra o laço na hora, ignorando o resto.

JavaScript

```
for (let i = 1; i <= 20; i++) {
```

```
if (i === 13) {  
  console.log("Número 13 encontrado! Parando o laço...");  
  break; // Sai do for imediatamente  
}  
console.log(i);  
}
```

5. Crie um array de produtos e use um laço para imprimir o nome de cada um deles.

Para percorrer arrays (listas), usamos o `length` (tamanho do array) para saber até onde o `for` deve ir. Lembre-se que os índices dos arrays começam no 0.

JavaScript

```
let produtos = ["Camiseta", "Tênis", "Jaqueta", "Mochila"];
```

```
for (let i = 0; i < produtos.length; i++) {  
  console.log("Produto:", produtos[i]);  
}
```

/* Dica extra: Existe um jeito mais moderno e fácil de fazer isso em JavaScript usando o 'for...of':

```
for (let produto of produtos) {  
  console.log("Produto:", produto);  
}  
*/
```

Exercícios de Fixação

1. Desafio: Radar de Velocidade

Usamos um bloco simples de `if/else` para avaliar a condição.

JavaScript

```
let velocidade = 85; // Altere este valor para testar
```

```
if (velocidade > 80) {  
  console.log("Multado!");  
} else {  
  console.log("Dentro do limite");  
}
```

2. Desafio: Contagem de Estoque

O `while` vai continuar rodando enquanto a variável `caixas` for maior que zero. A cada volta, diminuimos 1 (`caixas--`).

JavaScript

```
let caixas = 5;
```

```
while (caixas > 0) {  
  console.log(`Faltam ${caixas} caixas`);  
  caixas--;  
}
```

```
}  
console.log("Estoque zerado!");
```

3. Desafio: Validador de Tipos

A função `typeof` retorna uma string representando o tipo da variável. Vale lembrar que em JavaScript, tanto `null` quanto Arrays são avaliados como `"object"`.

JavaScript

```
let dado = "Olá Mundo"; // Troque por 10, [], null para testar
```

```
switch (typeof dado) {  
  case "string":  
    console.log("Tipo: Texto");  
    break;  
  case "number":  
    console.log("Tipo: Numérico");  
    break;  
  case "object":  
    console.log("Tipo: Objeto (Pode ser Array, Objeto ou Null)");  
    break;  
  default:  
    console.log("Tipo: Desconhecido");  
    break;  
}
```

4. Desafio: Caixa Eletrônico (Ternário)

O operador ternário `condição ? caso_verdadeiro : caso_falso` é excelente para resolver lógicas simples em uma única linha.

JavaScript

```
let saldo = 1000;  
let saqueDesejado = 250;  
  
let resposta = (saldo >= saqueDesejado) ? "Saque Autorizado" : "Saldo Insuficiente";  
console.log(resposta);
```

5. Desafio: Pular os Pares

O operador módulo `%` retorna o resto da divisão. Se o resto da divisão por 2 for zero, o número é par. O `continue` manda o laço ignorar o resto do bloco e ir direto para a próxima iteração.

JavaScript

```
for (let i = 1; i <= 20; i++) {  
  if (i % 2 === 0) {  
    continue; // Ignora os pares  
  }  
  console.log(i); // Imprime apenas os ímpares  
}
```

6. Desafio: O Menu do Dashboard

Aqui misturamos as estruturas. O `switch` decide a ação baseada no pedido, e dentro do caso correto, o `for` executa o trabalho repetitivo.

JavaScript

```
let pedido = "Relatorios";
```

```
switch (pedido) {
  case "Relatorios":
    console.log("Iniciando geração de relatórios:");
    for (let i = 1; i <= 3; i++) {
      console.log(`Gerando Relatorio ${i}...`);
    }
    break;
  case "Clientes":
    console.log("Carregando lista de clientes...");
    break;
  default:
    console.log("Opção inválida ou não implementada.");
    break;
}
```

7. Desafio: Limpador de Array de Dados (Truthy/Falsy)

Em JavaScript, valores como `""` (string vazia), `null`, `0`, `undefined` e `NaN` são avaliados como *Falsy* (falsos) quando colocados dentro de um `if`. Todo o resto é *Truthy*.

JavaScript

```
let listaSuja = ["João", "", null, "Maria", 0, "José"];
let usuariosValidos = [];
```

```
for (let i = 0; i < listaSuja.length; i++) {
  // Se o valor for "Truthy", ele entra no if
  if (listaSuja[i]) {
    usuariosValidos.push(listaSuja[i]);
  }
}
```

```
console.log(usuariosValidos); // Resultado: [ 'João', 'Maria', 'José' ]
```

8. Desafio: Loop Seguro e Auditoria com IA

Código com falha proposital (Não execute no seu navegador normal, pois vai travar a aba!):

JavaScript

```
// CUIDADO: LOOP INFINITO
```

```
let contador = 0;
while (true) {
  contador++;
  console.log(contador);
  // Faltou a condição de quebra (break) aqui!
}
```

Resumo da Explicação da IA:

"O seu navegador travou porque o laço `while (true)` nunca encontra uma condição para parar de executar. O JavaScript na web roda em uma única thread (fio de execução). Ao prender essa thread em um loop sem fim, o navegador não consegue fazer mais nada — não processa cliques, não renderiza a tela e consome 100% da capacidade do núcleo do processador alocado para aquela aba, até que o sistema operacional force a parada ou a memória se esgote."

9. Desafio: Escape de Matriz (Uso de Labels)

Rótulos (labels) permitem que você dê um nome ao seu laço. Assim, se você estiver num laço dentro de outro, pode usar o `break NomeDoRotulo` para encerrar todos de uma vez, e não apenas o laço interno.

JavaScript

```
let matriz = [
  ["Dado 1", "Dado 2"],
  ["Dado 3", "CORROMPIDO"],
  ["Dado 5", "Dado 6"]
];

let corrompido = false;

// Nomeamos o laço externo como "loopExterno"
loopExterno: for (let y = 0; y < matriz.length; y++) {
  for (let x = 0; x < matriz[y].length; x++) {

    console.log("Analisando:", matriz[y][x]);

    if (matriz[y][x] === "CORROMPIDO") {
      console.log("Erro crítico encontrado! Abortando todo o processo.");
      corrompido = true;
      break loopExterno; // Para os dois 'for' de uma vez só!
    }
  }
}
```

10. Desafio: Fatiamento de Usuários do Sistema

Vamos manter um contador de erros. Se o nome tiver menos de 3 caracteres, aumentamos o contador. Se esse contador chegar em 4, paramos tudo.

JavaScript

```
let lista = ["Oi", "Ana", "Lu", "Carlos", "Ze", "Edu"];
let errosEncontrados = 0;
let usuariosIrregulares = [];

for (let i = 0; i < lista.length; i++) {
  let nome = lista[i];

  if (nome.length < 3) {
    errosEncontrados++;
    usuariosIrregulares.push(nome);

    if (errosEncontrados === 4) {
      console.log("Lote muito defeituoso. Suspenso.");
      break; // Interrompe o loop no 4º erro
    }
  }
}
```



```
}  
}  
}
```

```
console.log(`Processo finalizado. Total de erros: ${errosEncontrados}`);  
console.log(`Usuários barrados:`, usuariosIrregulares);
```